



《面向对象程序设计》之——

运算符重载

连 盛

shenglian@fzu.edu.cn

计算机与大数据学院

福州大学

讲授内容

- 运算符重载的概念
- 以成员函数的方式重载运算符的方法
- 以友元函数的方式重载运算符的方法
- 流插入和流提取运算符的重载方法
- 一般单目和双目运算符的重载
- 赋值运算符重载的方法
- 类型转换运算符重载的方法

重载?

- 函数重载
- 运算符重载

函数的重载

所谓函数的重载是指完成不同功能的函数可以具有相同的函数名。

C++的编译器是根据函数的实参来确定应该调用哪一个函数的。

```
int fun(int a, int b)
```

```
{ return a+b; }
```

```
int fun (int a)
```

```
{ return a*a; }
```

```
void main(void)
```

```
{ cout<<fun(3,5)<<endl;
```

```
cout<<fun(5)<<endl;
```

```
}
```

8

25

函数重载

定义的重载函数必须具有**不同的参数个数**，或**不同的参数类型**。只有这样编译系统才有可能根据不同的参数去调用不同的重载函数。

编译器通过对比实参与函数参数列表的匹配来选择最合适的重载函数。

仅返回值不同时，不能定义为重载函数。

如果实参与某个重载函数的参数精确匹配，则会调用该函数；如果没有精确匹配但存在更广泛匹配的情况，则编译器会选择**最佳匹配**的重载函数。

函数重载：“最佳匹配”

假设我们有以下两个重载函数：

```
void printNumber(int x) {  
    std::cout << "Integer number: " << x << std::endl;  
}  
  
void printNumber(double y) {  
    std::cout << "Double number: " << y << std::endl;  
}
```

现在考虑以下函数调用：

```
printNumber(10); // 调用哪个函数？  
printNumber(5.5); // 调用哪个函数？  
printNumber('A'); // 调用哪个函数？
```

函数重载：“最佳匹配”

根据C++17标准，在调用重载函数时，编译器首先找出所有候选函数。然后，对每个实参，编译器尝试将其转换为候选函数的形参类型，形成转换序列，并根据一套规则计算每个函数的适配程度，最终选择适配程度最高的那个函数进行调用。具体来说，选择最佳匹配函数的规则包括但不限于：

- 如果有精确匹配（**Exact Match**），即实参类型与形参类型完全一致，无需任何类型转换，则优先选择。
- 如果没有精确匹配，编译器将考察每个可行函数（**Viable Function**），并计算从实参类型到形参类型的转换成本，选择转换成本最低的函数调用。 [参考链接: cppreference.com](http://cppreference.com)

函数重载：小结

- 完成不同功能的函数可以具有相同的函数名
- 根据实参（参数数量，参数类型）选择调用具体的重载函数
- 若参数类型不完全符合，则选择“最佳匹配”

运算符重载

```
int sum, a=3, b=2;
```

```
sum=a+b;      (int)=(int) + (int)
```

```
float add, x=3.2, y=2.5;
```

```
add=x+y;      (float)=(float) + (float)
```

```
char str[4], c1[2]="a", c2[2]="b";
```

```
str=c1+c2;     (char *)=(char *) + (char *)
```

系统自动
识别数据
类型

编译系统中的运算符“+”本身不能做这种运算，若使上式可以运算，必须重新定义“+”运算符，这种重新定义的过程成为运算符的重载。

```
class A
{ float x,y;
public:
    A(float a=0, float b=0){ x=a; y=b; }
};

void main(void)
{ A  a(2,3), b(3,4), c;

  c=a+b;
}
```

两对象不能使用+，必须重新定义+

运算符重载就是赋予已有的运算符多重含义。C++通过重新定义运算符，使它
能够用于特定类的对象执行特定的功能₁₁

运算符的重载从另一个方面体现了OOP技术的多态性，且同一运算符根据不同的运算对象可以完成不同的操作。

为了重载运算符，必须定义一个函数，并告诉编译器，遇到这个重载运算符就调用该函数，由这个函数来完成该运算符应该完成的操作。这种函数称为运算符重载函数，它通常是类的成员函数或者是友元函数。运算符的操作数通常也应该是类的对象。

运算符重载的概念 (1/2)

- 类似于函数重载
- 把传统的运算符用于用户自定义的对象
- 直观自然，可以提高程序的可读性
- 体现了C++的多态性和可扩充性
- 通过定义名为 `operator<运算符>` 的函数来实现运算符重载

重载为类的成员函数

格式如下：

关键字

运算的对象

<类名> **operator**<运算符>(<参数表>)

{ 函数体 }

返回类型

函数名

运算的对象

A **operator +** (A &); //重载了类A的“+”运算符

其中：**operator**是定义运算符重载函数的关键字，
与其后的运算符一起构成函数名。

没有重载运算符的例子

```

class A
{
    int i;

public:
    A(int a=0) { i=a; }

    void Show(void){ cout<<"i="<<i<<endl; }

    void AddA(A &a, A &b) //利用函数进行类之间的运算
    {
        i=a.i+b.i;
    }
};

void main(void)
{
    A a1(10),a2(20),a3;
    a1.Show ();
    a2.Show ();

    // a3=a1+a2; //不可直接运算
    a3.AddA(a1,a2); //调用专门的功能函数
    a3.Show ();
}
    
```

利用函数完成了加法运算

用和作对象调用函数

```

class A
{
    int i;
public:
    A(int a=0){ i=a; }
    void Show(void){ cout<<"i="<<i<<endl; }
    void AddA(A &a, A &b) //利用函数进行类之间的运算
    {
        i=a.i+b.i;
    }
    A operator+(A &a) //重载运算符+
    {
        A t; t.i=i+a.i; return t;
    }
};

void main(void)
{
    A a1(10),a2(20),a3;
    a1.Show ();
    a2.Show ();
    a3=a1+a2; //重新解释了加法，可以直接进行类的运算
    a3.AddA(a1,a2); //调用专门的功能函数
    a3.Show ();
}

```

相当于a3=a1.operator+(a2)

重载运算符与一般函数的比较:

相同: 1) 均为类的成员函数; 2) 实现同一功能

返回值 **函数名** **形参列表**

```
void AddA(A &a, A &b)
{   i=a.i+b.i; }
```

函数调用:

a3.AddA(a1,a2);

由对象a3调用

返回值 **函数名**

```
A operator +(A &a)
{   A   t;
    t.i=i+a.i;
    return t;
}
```

形参

函数调用:

a3=a1+a2;

a3=a1.operator+(a2);

由对象a1调用

返回值

函数名

A operator +(A &a)

{ A t;

t.i=i+a.i;

return t;

}

形参

函数调用:

a3=a1+a2;

a3=a1.operator+(a2);

由对象a1调用

总结:

重新定义运算符，由左操作数调用重载的运算符（作为成员函数），并将右操作数作为参数传递给该函数。最后将函数返回值赋给运算结果的对象。

例1：复数的 +、-、= 运算 (1/5)

```
// 文件1: complex1.h——复数类的定义
#ifndef __COMPLEX1_H
#define __COMPLEX1_H
class Complex
{public:
    Complex(double = 0.0, double = 0.0);
    Complex operator+(const Complex&) const;
    Complex operator-(const Complex&) const;
    Complex& operator=(const Complex&);
    void print() const;
private:
    double real;           // real part
    double imaginary;      // imaginary part
};
#endif
```

“最低访问权”原则

例1：复数的 +、-、= 运算 (2/5)

//文件2: complex1.cpp——复数类的成员函数定义

```
#include <iostream>
```

```
#include "complex1.h"
```

```
Complex::Complex(double r, double i)
```

```
{    real = r;
```

```
    imaginary = i;
```

```
}
```

```
Complex Complex::operator+(const Complex  
                           & operand2) const
```

```
{Complex sum;
```

```
    sum.real = real + operand2.real;
```

```
    sum.imaginary = imaginary +  
                      operand2.imaginary;
```

```
    return sum;
```

```
}
```

例1 : 复数的 +、-、= 运算 (3/5)

```
Complex Complex::operator-(const Complex  
                           & operand2) const
```

```
{Complex diff;  
  diff.real = real - operand2.real;  
  diff.imaginary = imaginary -  
                  operand2.imaginary;  
  return diff; }
```

```
Complex& Complex::operator=(const  
                             Complex & right)
```

```
{real = right.real;  
  imaginary = right.imaginary;  
  return *this; }
```

```
void Complex::print( ) const
```

```
{cout<<' ('<<real<< " , " << imaginary  
  << ') ' ; }
```

例1：复数的 +、-、= 运算 (4/5)

```
//文件3: FIG13_1.cpp——主函数定义
#include <iostream>
#include "complex1.h"
main()
{Complex x, y(4.3, 8.2), z(3.3, 1.1);
  cout << "x: ";
  x.print();
  cout << "\ny: ";
  y.print();
  cout << "\nz: ";
  z.print();
  x = y + z;
  cout << "\n\nx = y + z:\n";
```

例1：复数的 +、-、= 运算 (5/5)

```
x.print();  
cout << " = ";  
y.print();  
cout << " + ";  
z.print();  
x = y - z;  
cout << "\n\nx = y - z:\n";  
x.print();  
cout << " = ";  
y.print();  
cout << " - ";  
z.print();  
cout << '\n';  
return 0;}
```

程序执行结果:

x: (0, 0)

y: (4.3, 8.2)

z: (3.3, 1.1)

x = y + z:

$$(7.6, 9.3) = (4.3, 8.2) + (3.3, 1.1)$$

x = y - z:

$$(1, 7.1) = (4.3, 8.2) - (3.3, 1.1)$$

运算符重载的概念 (2/2)

- C++的所有运算符都可以被重载吗?
 - 包括双目算术运算符、关系、逻辑、单目、自增自减、位运算符、赋值运算符、空间申请与释放、指针访问成员→ 等，都能被重载
 - . * :: ?: sizeof 不能被重载
- 原因？

运算符重载的概念 (2/2)

- 什么情况下需要考虑运算符重载?
 - 当要把自定义类的对象用作运算符的操作数时
- 运算符重载函数如何定义?
 - 保持运算符的原有属性，作为类的成员函数或友元函数、作为一般函数
 - 特例：`( ) [ ] -> =` 及类型转换运算符的重载

运算符重载：注意

- C++的所有运算符都可以被重载吗？
 - `.` `*` `::` `?:` `sizeof` 不能被重载
- 重载不能改变操作数的个数（单目、双目）
- 重载不能改变运算符优先级
- 不能改变结合性
- 不能有默认参数

运算符重载：成员函数与友元函数 (1/3)

- 运算符的重载函数定义为类的**成员函数**
 - 当该成员函数不带参数时，支持以该类对象为唯一操作数的运算
 - 当该成员函数带一个参数时，支持以该类对象为左操作数、以所带参数类型的数据为右操作数的运算（比如复数加）

重载为类的成员函数

格式如下：

关键字

运算的对象

<类名> **operator**<运算符>(<参数表>)

{ 函数体 }

返回类型

函数名

运算的对象

A **operator +** (A &); //重载了类A的“+”运算符

其中：**operator**是定义运算符重载函数的关键字，
它与其后的运算符一起构成函数名。

没有重载运算符的例子

```

class A
{
    int i;

public:
    A(int a=0) { i=a; }

    void Show(void){ cout<<"i="<<i<<endl; }

    void AddA(A &a, A &b) //利用函数进行类之间的运算
    {
        i=a.i+b.i;
    }
};

void main(void)
{
    A a1(10),a2(20),a3;
    a1.Show ();
    a2.Show ();

    // a3=a1+a2; //不可直接运算
    a3.AddA(a1,a2); //调用专门的功能函数
    a3.Show ();
}
    
```

利用函数完成了加法运算

用和作对象调用函数

```

class A
{
    int i;
public:
    A(int a=0){ i=a; }
    void Show(void){ cout<<"i="<<i<<endl; }
    void AddA(A &a, A &b) //利用函数进行类之间的运算
    {
        i=a.i+b.i;
    }
    A operator +(A &a) //重载运算符+
    {
        A t; t.i=i+a.i; return t;
    }
};

void main(void)
{
    A a1(10),a2(20),a3;
    a1.Show ();
    a2.Show ();
    a3=a1+a2; //重新解释了加法，可以直接进行类的运算
    a3.AddA(a1,a2); //调用专门的功能函数
    a3.Show ();
}

```

相当于a3=a1.operator+(a2)

用成员函数实现运算符的重载时，**运算符的左操作数为当前对象**，并且要用到隐含的this指针。**运算符重载函数不能定义为静态的成员函数。Why?**

运算符重载：成员函数与友元函数 (2/3)

- 运算符的重载函数定义为类的友元函数
 - 当该函数带一个参数时，支持以该参数类型的对象为唯一操作数的运算
 - 当该函数带两个参数时，支持以第一个参数类型的数据为左操作数、以第二个参数类型的数据为右操作数的运算

运算符重载：成员函数与友元函数 (3/3)

- 在包含对象的双目运算中，当双目运算符的左操作数不是某类的对象时，重载函数不能定义为该类的成员函数，只能通过友元函数
- 运算符重载函数的定义要保证运算无二义性

运算符重载为友元函数

运算符重载为**成员函数**时，是由一个操作数调用另一个操作数。

A a, b, c;

c=a+b; 实际上是**c=a.operator+(b);**

c=++a; 实际上是**c=a.operator++();**

c+=a; 实际上是**c.operator+=(a);**

重载+=

即函数的实参只有一个或没有。

友元函数是在类外的普通函数，与一般函数的区别是可以调用类中的私有或保护数据。

将运算符的重载函数定义为友元函数，参与运算的对象全部成为函数参数。

A a, b, c;

c=a+b; 实际上是 **c=operator+(a, b);**

c=++a; 实际上是 **c=operator++(a);**

c+=a; 实际上是 **operator+=(c, a);**

单目运算符重载 (1/3)

- 操作数是自定义类的对象或对象的引用
- 作为成员函数重载
 - 没有参数
- 作为友元函数重载
 - 参数为自定义类的对象或对象的引用

例2: 定义字符串类String, 并以成员函数的方式重载运算符! 以判断对象中的字符串是否为空串 (1/3)

```
//文件string.h
#ifndef __STRING__H__
#define __STRING__H__
#include <stdlib.h>
class String
{public:
    String(char *m="");
    ~String();
    //定义运算符重载成员函数原型
    bool operator! ( );
private:
    char *str;
};
#endif
```

例2: 定义字符串类String, 并以成员函数的方式重载运算符 ! 以判断对象中的字符串是否为空串 (2/3)

```
//文件String.cpp
#include <string.h>
#include "string.h"
String::String(char *m)
{str = new char[strlen(m)+1];
  strcpy(str,m);
}
String::~~String()
{delete [] str;}
bool String::operator! () //实现运算符重载
{if (strlen(str) == 0)
  return true;
  return false;
}
```


例2: 定义字符串类String, 并以成员函数的方式重载运算符 ! 以判断对象中的字符串是否为空串 (3/3)

```
//文件ex13_1.cpp
#include <iostream.h>
#include "String.h"
main( )
{String s1, s2("some string");
  if (!s1)
    cout<<"s1 is NULL!"<<endl;
  else
    cout<<"s1 is not NULL!"<<endl;
  if (!s2)
    cout<<"s2 is NULL!"<<endl;
  else
    cout<<"s2 is not NULL!"<<endl;
  return 0;
}
```

程序执行结果:

s1 is NULL!

s2 is not NULL!

例3: 定义字符串类String, 并以友元函数的方式重载运算符 ! 以判断对象中的字符串是否为空串 (1/3)

```
//文件string.h
```

```
#if !defined __STRING__H__
```

```
#define __STRING__H__
```

```
#include <stdlib.h>
```

```
class String
```

```
{//定义运算符重载友元函数原型
```

```
    friend bool operator! (String &s) ;
```

```
    public:
```

```
        String(char *m="") ;
```

```
        ~String() ;
```

```
    private:
```

```
        char *str;
```

```
};
```

```
#endif
```

例3: 定义字符串类String, 并以友元函数的方式重载运算符 ! 以判断对象中的字符串是否为空串 (2/3)

//文件String.cpp

```
#include <string.h>
```

```
#include "string.h"
```

```
String::String(char *m)
```

```
{str = new char[strlen(m)+1];
```

```
strcpy(str,m);
```

```
}
```

```
String::~~String()
```

```
{delete [] str;}
```

```
bool operator!(String &s) //实现友元函数
```

```
{if (strlen(s.str) == 0)
```

```
return true;
```

```
return false;
```

```
}
```

例3: 定义字符串类String, 并以友元函数的方式重载运算符 ! 以判断对象中的字符串是否为空串 (3/3)

```
//文件ex13_2.cpp
#include <iostream.h>
#include "String.h"
main()
{String s1, s2("some string");
  if (!s1) //括号中等价于operator!(s1)
    cout<<"s1 is NULL!"<<endl;
  else
    cout<<"s1 is not NULL!"<<endl;
  if (!s2)
    cout<<"s2 is NULL!"<<endl;
  else
    cout<<"s2 is not NULL!"<<endl;
  return 0;
}
```

程序执行结果:

s1 is NULL!

s2 is not NULL!

运算符重载为成员函数/友元函数

- 运算符重载为**成员函数**时，是由一个操作数调用另一个操作数。
- 将运算符的重载函数定义为友元函数，**参与运算的对象全部成为函数参数**。



a++?

	重载为成员函数	定义为友元函数
c=a+b;	c=a.operator+(b);	c=operator+(a, b);
c=++a;	c=a.operator++();	c=operator++(a);
c+=a;	c.operator+=(a);	operator+=(c, a);

- 参数传递方式：**成员函数重载时，左操作数（对于双目运算符）作为调用对象隐式传递（this）；友元函数重载需要显式传递左右操作数。
- 访问权限：**成员函数重载可以直接访问类的私有成员；友元函数虽然也可以访问类的私有成员，但并不属于类的成员函数，因此没有直接的访问权限。

```
class Complex {
public:
    double real, imaginary;
    // 构造函数以及其他必要方法...
    // 重载+运算符    采用成员函数版本
    Complex operator+(const Complex& other) const {
        return Complex(real + other.real, imaginary +
other.imaginary);
    }
    // 或者采用非成员函数版本，这样两边的操作数地位更加对称
    friend Complex operator+(const Complex& lhs,
const Complex& rhs) {
        return Complex(lhs.real + rhs.real,
lhs.imaginary + rhs.imaginary);
    }
};
```


// 使用时，无论哪种顺序调用加法，结果都应该相同

```
Complex a(1.0, 2.0);
```

```
Complex b(3.0, 4.0);
```

```
Complex sum1 = a + b;
```

```
Complex sum2 = b + a; // 交换了加法操作数的位置，但结  
    果应与sum1一致
```

```
assert(sum1 == sum2); // 如果满足交换律，此断言不会失  
    败
```

单目运算符重载 (2/3)

■ 特殊的单目运算符++、--

以++ 为例（-- 同理），设b为类C的对象

■ 前自增，如：++b

- 重载函数可为C的成员函数，原型为
返回值类型 operator++();

- 重载函数也可为C的友元函数，原型为
返回值类型 operator++(C &);

单目运算符重载 (3/3)

■ 特殊的单目运算符++、--

以++ 为例（-- 同理），设b为类C的对象

■ 后自增，如：b++

- 重载函数可为C的成员函数，原型为

返回值类型 `operator++(int);`

b++转换为函数调用**`b.operator++(0)`**

- 重载函数也可为C的友元函数，原型为

返回值类型 `operator++(C &, int);`

b++转换为函数调用**`operator++(b, 0)`**

Why?



```

1  void BigNum::operator++() {
2      *this = *this + 1;
3  }
4
5  int main() {
6      BigNum num1;
7      cin >> num1;
8      num1++;    // (1)
9      cout << num1 << endl;
10     ++num1;    // (2)
11     cout << num1 << endl;
12     return 0;
13 }

```

error: no 'operator++(int)' declared for postfix '++' [-fpermissive]

Why?



- 由于编译器必须能够识别出前缀自增与后缀自增，人为规定用 `operator++()` 重载前置运算符，用 `operator++(int)` 重载后置运算符。
- 在这这里的 `int` 并没有什么实际的意义，仅仅是为了区分重载的是前置的形式还是后置的形式。

讲授内容

- 运算符重载的概念
- 以成员函数的方式重载运算符的方法
- 以友元函数的方式重载运算符的方法
- 流插入和流提取运算符的重载方法
- 一般单目和双目运算符的重载
- 赋值运算符重载的方法
- 类型转换运算符重载的方法

重载流插入和流提取运算符

■ printf/scanf vs cout/cin ?

```
void func(int i, float f){  
    printf( "%d %d\n" , i, f);}
```

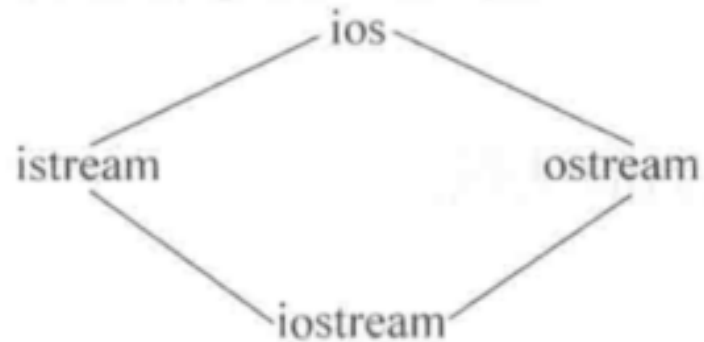
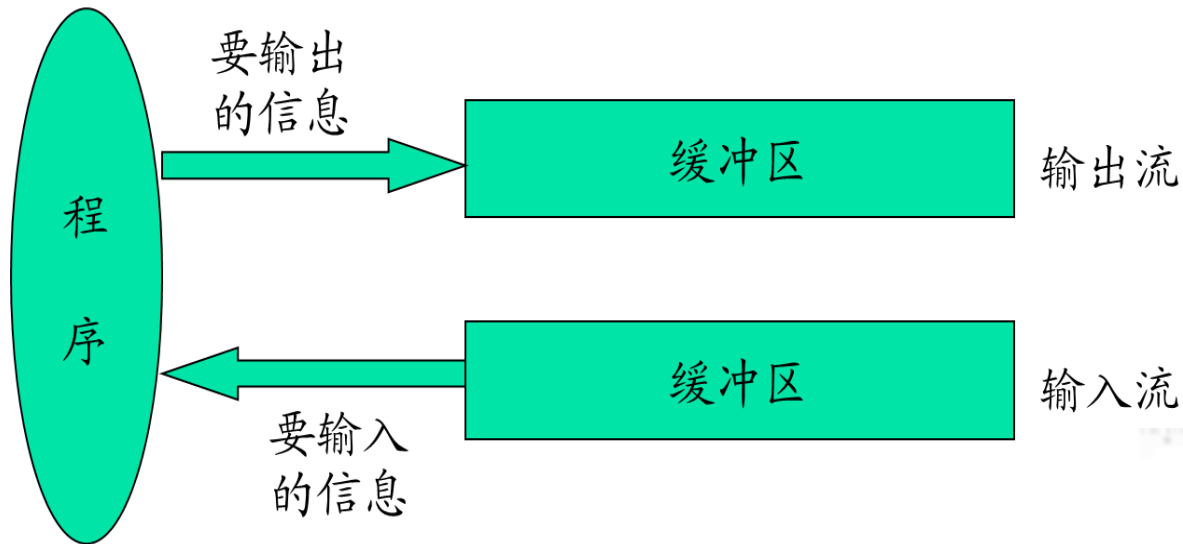
```
scanf( "%d" , &i);    ?
```

```
scanf( "%d" , i);    ?
```

可能造成严重的后果!

重载流插入和流提取运算符

➤ **C++** 中把数据之间的传输操作称为流。



重载流插入和流提取运算符

- 目的：语句 `cout << "string";` 能在标准输出设备上输出字符串 `string`。
`cout` 是类 `ostream` 的一个对象（定义于头文件 `ostream.h` 中）

如何实现：

```
String s("string");  
cout << s << endl;
```

例4: 重载流插入运算符<<和流提取运算符>>, 实现用cout和cin直接输出和输入类String的对象 (1/4)

```
//文件string.h
#ifndef __STRING__H__
#define __STRING__H__
#include <iostream>
class String
{friend ostream & operator<<(ostream
                        &output, String &s);
  friend istream & operator>>(istream
                        &input, String &s);
public:
  String(char *m=" ");
  ~String();
private:
  char *str;
}; #endif
```

例4: 重载流插入运算符<<和流提取运算符>>, 实现
用cout和cin直接输出和输入类String的对象 (2/4)

//文件string.cpp

```
#include <string.h>
```

```
#include "string.h"
```

```
String::String(char *m)
```

```
{str = new char[strlen(m)+1];
```

```
    strcpy(str,m);
```

```
}
```

```
String::~~String()
```

```
{
```

```
    delete [] str;
```

```
}
```

例4: 重载流插入运算符<<和流提取运算符>>, 实现用cout和cin直接输出和输入类String的对象 (3/4)

```
//续文件string.cpp, 定义运算符<<和>>的重载函数
ostream & operator<<(ostream &output,
                    String &s)

{output << s.str;
 return output;
}

istream & operator>>(istream &input,
                    String &s)

{char temp[1000];
 input >> temp;
 delete [] s.str;
 s.str = new char[strlen(temp)+1];
 strcpy(s.str,temp);
 return input;
}
```

例4: 重载流插入运算符<<和流提取运算符>>, 实现
用cout和cin直接输出和输入类String的对象 (4/4)

//文件ex13_3.cpp

```
#include "string.h"
```

```
main()
```

```
{String s1,s2;
```

```
    cout << "Please input two strings: "
        << endl;
```

```
    cin >> s1 >> s2;
```

```
    cout << "Output is: " << endl;
```

```
    cout << "s1 -- " << s1
```

```
        << endl << "s2 -- " << s2 << endl;
```

```
    return 0;
```

```
}
```

程序执行结果:

Please input two strings:

Fuzhou

Hangzhou

Output is:

s1 -- Fuzhou

s2 -- Hangzhou

注意:

- 凡是用“cout<<”和“cin>>”对标准类型数据进行输入输出的，都要用
`#include <iostream>`把头文件包含到本程序文件中。
- 为什么只能将重载>>和<<的函数作为友元函数？

如果想直接用“<<”和“>>”输出和输入自己声明的类型的的数据，必须对它们重载。

对“<<”和“>>”重载的函数形式如下：

```
istream & operator >> (istream &, 自定义类 &);
```

```
ostream & operator << (ostream &, 自定义类 &);
```

- 重载运算符“>>”的函数的第一个参数和函数的类型都必须是istream&类型，第二个参数是要进行输入操作的类。
- 重载“<<”的函数的第一个参数和函数的类型都必须是ostream&类型，第二个参数是要进行输出操作的类。
- 只能将重载“>>”和“<<”的函数作为友元函数或普通的函数，而不能将它们定义为成员函数。

区分什么情况下的“<<”是标准类型数据的流插入符，什么情况下的“<<”是重载的流插入符。如：

```
Complex c3(3, 5);    //Complex类中<<已重载  
cout<<c3<<5<<endl;
```

第一个调用重载的流插入符，后面两个“<<”不是重载的流插入符，因为它的右侧不是Complex类对象而是标准类型的数据，是用预定义的流插入符处理的。

运算符重载中使用引用 (reference) 的重要性:

- 利用引用作为函数的形参可以在调用函数的过程中通过传址方式使形参成为实参的别名，因此不生成临时变量(实参的副本)，减少了时间和空间的开销
- 如果重载函数的返回值是对象的引用时，返回的不是常量，而是引用所代表的对象，它可以出现在赋值号的左侧而成为左值(left value)，可以被赋值或参与其他操作(如保留cout流的当前值以便能连续使用“<<”输出)。
- 使用引用时要特别小心，因为修改了引用就等于修改了它所代表的对象。

一般双目运算符重载

- 一般双目运算符的重载要考虑两个因素：
 - 被重载运算符的左操作数是什么数据类型
 - 是否要保留运算符的可交换性（只针对具有可交换性的运算符而言）
- 双目运算符的重载可实现为
 - 带有一个参数的（非静态）成员函数（左操作数必须为该类的对象或对象的引用）或
 - 带有两个参数的非成员函数（其参数之一必须是类的对象或对象的引用）

例5: 重载运算符+=使x+=y实现将对象y中的字符串连接到对象x的字符串后面, 连接后的x作为运算结果 (1/5)

```
//文件string.h, 定义类String
#ifndef __STRING__H__
#define __STRING__H__
#include <iostream>
class String
{friend ostream & operator<<(ostream
                        &output, String &s);
  friend istream & operator>>(istream
                        &input, String &s);
public:
  String(char *m="");
  ~String();
  String & operator+=(String &s);
private:
  char *str;}; #endif
```



例5: 重载运算符+=使x+=y实现将对象y中的字符串连接到对象x的字符串后面, 连接后的x作为运算结果 (2/5)

//文件string.cpp, 类String的实现

```
#include <string.h>
```

```
#include "string.h"
```

```
String::String(char *m)
```

```
{str = new char[strlen(m)+1];
```

```
    strcpy(str,m);
```

```
}
```

```
String::~~String()
```

```
{delete [] str;}
```

//定义运算符<<的重载函数

```
ostream & operator<<(ostream &output,  
                    String &s)
```

```
{output<<s.str;
```

```
    return output;
```

```
}
```



例5: 重载运算符+=使x+=y实现将对象y中的字符串连接到对象x的字符串后面, 连接后的x作为运算结果 (3/5)

```
//续文件string.cpp
```

```
//定义运算符>>的重载函数
```

```
istream & operator>>(istream &input,  
                      String &s)
```

```
{char temp[1000];
```

```
  input>>temp;
```

```
  delete []s.str;
```

```
  s.str = new char[strlen(temp)+1];
```

```
  strcpy(s.str,temp);
```

```
  return input;
```

```
}
```



例5: 重载运算符+=使x+=y实现将对象y中的字符串连接到对象x的字符串后面, 连接后的x作为运算结果 (4/5)

```
//续文件string.cpp
```

```
//定义运算符+=的重载函数
```

```
String &String::operator+=(String &s)
```

```
{char temp[4096];
```

```
    strcpy(temp, str);
```

```
    strcat(temp, s.str);
```

```
    delete []str;
```

```
    str = new char[strlen(temp)+1];
```

```
    strcpy(str, temp);
```

```
    return *this;
```

```
}
```




例5: 重载运算符+=使x+=y实现将对象y中的字符串连接到对象x的字符串后面, 连接后的x作为运算结果 (5/5)

```
//文件ex13_4.cpp, 测试重载的运算符
#include "string.h"
main()
{String s1,s2;
  cin>>s1>>s2; //输入字符串Fuzhou和Hangzhou
  cout<<"s1  --  "<<s1<<endl;
  s1+=s2;
  cout<<"s1  after  s1+=s2  --  "<<s1<<endl;
  return 0;
}
```

程序执行结果:

Fuzhou

Quanzhou

s1 -- Fuzhou

s1 after s1+=s2 -- FuzhouQuanzhou

用友元函数实现运算符 += 的重载

```
class String { //声明友元函数operator+=  
friend String & operator+=(String &x,  
String &y); .....};
```

//运算符重载函数operator+=的实现

```
String & operator+=(String &x, String &y)  
{char temp[4096];  
strcpy(temp, x.str);  
strcat(temp, y.str);  
delete [] x.str;  
x.str = new char[strlen(temp)+1];  
strcpy(x.str, temp); return x;  
}
```

赋值运算符重载 (1/2)

- 赋值运算符可直接用在自定义类的对象赋值，其默认操作是逐个拷贝对象的所有数据成员
- 如果对象中包含动态分配的空间时，这种赋值方式就有可能出错，如

```
String s1("abc"), s2("def"), &s3=s1;  
s1 = s2;
```

对象s1和s2中都只有一个数据成员str，在赋值操作后，对象s1和s2的指针str都指向了同一块数据空间（即存放了"def"的存储空间），其问题是：

赋值运算符重载 (2/2)

- 另一块存放"abc"的空间就被遗弃了，将无法被访问和释放。
- 如果某个时刻对象s1被撤销，其析构函数将释放其指针str所指向的数据空间“def”（也就是对象s2的指针str所指向的数据空间），程序再访问对象s2的数据、或者撤销对象s2时都会发生指针错误。
- 当对象中包含动态分配的空间时，赋值运算符需要重载以实现同类对象赋值运算的正确和安全

例6: 重载运算符+和=, 使得类String的对象x、y、z
可以执行表达式 $x = y + z$ (1/6)

```
//文件string.h, 定义类String
#ifndef __STRING__H__
#define __STRING__H__
#include <iostream.h>
class String
{friend ostream & operator<<(
    ostream &output, String &s);
    friend istream & operator>>(
        istream &input, String &s);
public:
    String(char *m="");
    ~String();
    String & operator=(String &s);
    String & operator+(String &s);
private:    char *str; }; #endif
```

例6: 重载运算符+和=, 使得类String的对象x、y、z
可以执行表达式 $x = y + z$ (2/6)

//文件string.cpp, 类String的实现

```
#include <string.h>
```

```
#include "string.h"
```

```
String::String(char *m)
```

```
{str = new char[strlen(m)+1];
```

```
    strcpy(str,m);
```

```
}
```

```
String::~~String()
```

```
{delete [] str; }
```

```
ostream & operator<< (
```

```
    ostream &output, String &s)
```

```
{output<<s.str;
```

```
    return output;
```

```
}
```

例6: 重载运算符+和=, 使得类String的对象x、y、z
可以执行表达式 $x = y + z$ (3/6)

```
//续文件string.cpp
//定义运算符>>重载函数
istream & operator>>(
    istream &input, String &s)
{
    char temp[1000];
    cin>>temp;
    delete [] s.str;
    s.str = new char[strlen(temp)+1];
    strcpy(s.str, temp);
    return input;
}
```


例6: 重载运算符+和=, 使得类String的对象x、y、z
可以执行表达式 $x = y + z$ (4/6)

//续文件string.cpp

```
String &String::operator=(String &s)
```

```
{
```

```
    if (&s == this) //检查是否自我赋值, 非常重要  
        return *this;
```

```
    delete []str;    //释放当前对象的数据空间
```

```
    str = new char[strlen(s.str)+1];
```

```
                    //重新分配适当大小的空间
```

```
    strcpy(str, s.str);
```

```
    return *this;
```

```
}
```

例6: 重载运算符+和=, 使得类String的对象x、y、z
可以执行表达式 $x = y + z$ (5/6)

//续文件string.cpp

```
String &String::operator+(String &s)
{
    static String res;
    char temp[4096];
    strcpy(temp, str);
    strcat(temp, s.str);
    delete [] res.str;
    res.str = new char[strlen(temp)+1];
    strcpy(res.str, temp);
    return res;
}
```

例6: 重载运算符+和=, 使得类String的对象x、y、z
可以执行表达式 $x = y + z$ (6/6)

//文件ex13_5.cpp, 测试重载的运算符

```
#include "String.h"
```

```
main()
```

```
{String s1,s2;
```

```
    cout << "Please input two strings: "
         << endl;
```

```
    cin >> s1 >> s2;
```

```
    cout << "Output is: " << endl;
```

```
    cout << "s1 -- " << s1 << endl;
```

```
    cout << "s2 -- " << s2 << endl;
```

```
    s1 = s1 + s2;
```

```
    cout << "after s1 = s1 + s2;" << endl;
```

```
    cout << "s1 -- " << s1 << endl;
```

```
    return 0;
```

```
}
```

程序执行结果:

Please input two strings:

Fuzhou

Hangzhou

Output is:

s1 -- Fuzhou

s2 -- Hangzhou

after s1 = s1 + s2;

s1 -- FuzhouHangzhou

不同类型数据间的转换

- 隐式类型转换
- 显式类型转换

在C++中，某些不同类型数据之间可以自动转换，例如：

```
int i = 6;  
i = 7.5 + i;
```

```
int    double    double
```

这种转换是由C++编译系统自动完成的，用户不需干预。这种转换称为隐式类型转换。

C++还提供显式类型转换，程序人员在程序中指定将一种指定的数据转换成另一指定的类型，其形式为

类型名(数据) 如:

```
int (89.5);
```

其作用是将89.5转换为整型数89。

对于用户自己声明的类型，编译系统并不知道怎样进行转换。解决这个问题关键是让编译系统知道怎样去进行这些转换，需要定义专门的函数来处理

转换构造函数

转换构造函数 (conversion constructor function) 的作用是将一个其他类型的数据转换成一个类的对象。

先回顾一下以前学习过的几种构造函数：

默认构造函数。以Complex类为例，函数原型的形式为：

`Complex ( );` //没有参数

用于初始化的构造函数。函数原型的形式为：

`Complex(double r, double i);` //形参表列中一般有两个以上参数

用于复制对象的复制构造函数。函数原型的形式为：


```
Complex (Complex &c);
```

//形参是本类对象的引用

现在又要介绍一种新的构造函数——**转换构造函数**。

转换构造函数只有一个形参，如

```
Complex(double r) {real=r; imag=0;}
```

其作用是将double型的参数r转换成Complex类的对象，将r作为复数的实部，虚部为0。用户可以根据需要定义转换构造函数，在函数体中告诉编译系统怎样去进行转换。

在类体中，可以有转换构造函数，也可以没有转换构造函数，视需要而定。以上几种构造函数可以同时出现在同一个类中，它们是构造函数的重载。编译系统会根据建立对象时给出的实参的个数与类型选择形参与之匹配的构造函数。

使用转换构造函数将一个指定的数据转换为类对象的方法如下：

- 先声明一个类。
- 在这个类中定义一个只有一个参数的构造函数，参数的类型是需要转换的类型，在函数体中指定转换的方法。
- 在该类的作用域内可以用以下形式进行类型转换：
类名(指定类型的数据)
就可以将指定类型的数据转换为此类的对象。

不仅可以将一个标准类型数据转换成类对象，也可以将另一个类的对象转换成转换构造函数所在的类对象。如可以将一个学生类对象转换为教师类对象，可以在Teacher类中写出下面的转换构造函数：

```
Teacher (Student& s)
```

```
{num=s.num; strcpy (name, s.name); sex=s.sex; }
```

注意： 对象s中的num, name, sex必须是**公有成员**，否则不能被类外引用。

类型转换函数

用转换构造函数可以将一个指定类型的数据转换为类的对象。但是不能反过来将一个类的对象转换为一个其他类型的数据（例如将一个Complex类对象转换成double类型数据）。

C++提供类型转换函数 (type conversion function) 来解决这个问题。类型转换函数的作用是将一个类的对象转换成另一类型的数据。如果已声明了一个Complex类，可以在Complex类中这样定义类型转换函数：

```
operator double()  
{return real;}
```

类型转换函数的一般形式为：

`operator 类型名 ( )`

`{实现转换的语句}`

- 在函数名前面不能指定函数类型，函数没有参数。
- 其返回值的类型是由函数名中指定的类型名来确定的。
- **类型转换函数只能作为成员函数**，因为转换的主体是本类的对象。不能作为友元函数或普通函数。

转换构造函数和类型转换运算符有一个共同的功能：
当需要的时候，编译系统会自动调用这些函数，建立一个无名的临时对象(或临时变量)。

使用类型转换函数的简单例子。

```
#include <iostream>
```

```
using namespace std;
```

```
class Complex
```

```
{public:
```

```
Complex( ){real=0;imag=0;}
```

```
Complex(double r,double i){real=r;imag=i;}
```

```
operator double( ) {return real;}
```

//类型转换函数

```
private:
```

```
double real;
```

```
double imag;
```

```
};
```

```
int main( )  
{Complex c1(3,4),c2(5,-10),c3;  
double d;  
d=2.5+c1; //要求将一个double数据与Complex类数据相加  
cout<<d<<endl;  
return 0;  
}
```

分析:

(1) 如果在Complex类中没有定义类型转换函数operator double, 程序编译将出错。

(2) 如果在main函数中加一个语句:

```
c3=c2;
```

由于赋值号两侧都是同一类的数据, 是可以合法进行赋值的, 没有必要把c2转换为double型数据。

(3) 如果在Complex类中声明了重载运算符 “+” 函数作为友元函数:

```
Complex operator+ (Complex c1,Complex c2) //定义运算符 “+” 重载函数
```

```
{return Complex(c1.real+c2.real, c1.imag+c2.imag);}
```


若在main函数中有语句

```
c3=c1+c2;
```

由于已对运算符“+”重载，使之能用于两个Complex类对象的相加，因此将c1和c2按Complex类对象处理，相加后赋值给同类对象c3。

如果改为

```
d=c1+c2; //d为double型变量
```

将c1与c2两个类对象相加，得到一个临时的Complex类对象，由于它不能赋值给double型变量，而又有对double的重载函数，于是调用此函数，把临时类对象转换为double数据，然后赋给d。

从前面的介绍可知：对类型的重载和本章开头所介绍的对运算符的重载的概念和方法都是相似的。重载函数都使用关键字operator。

因此，通常把类型转换函数也称为类型转换运算符函数，由于它也是重载函数，因此也称为类型转换运算符重载函数(或称强制类型转换运算符重载函数)。

假如程序中需要对一个Complex类对象和一个double型变量进行 $+$ 、 $-$ 、 $*$ 、 $/$ 等算术运算，以及关系运算和逻辑运算，如果不用类型转换函数，就要对多种运算符进行重载，以便能进行各种运算。这样，是十分麻烦的，工作量较大，程序显得冗长。如果用类型转换函数对double进行重载(使Complex类对象转换为double型数据)，就不必对各种运算符进行重载，因为Complex类对象可以被自动地转换为double型数据，而标准类型的数据的运算，是可以使用系统提供的各种运算符的。

包含转换构造函数、运算符重载函数和类型转换函数的程序。

```
#include <iostream>
using namespace std;
class Complex
{public:
    Complex( ){real=0;imag=0;}           //默认构造函数
    Complex(double r){real=r;imag=0;}    //转换构造函数
    Complex(double r,double i){real=r;imag=i;}
    friend Complex operator + (Complex c1,Complex c2);
    void display( );
private:
    double real;
    double imag;};
```

Complex operator + (Complex c1,Complex c2) //定义运算符
 “+” 重载函数

```
{return Complex(c1.real+c2.real, c1.imag+c2.imag);}
```

```
void Complex::display( )
```

```
{cout<<"("<<real<<","<<imag<<"i)"<<endl;}
```

```
int main( )
```

```
{Complex c1(3,4),c2(5,-10),c3;
```

```
c3=c1+2.5;
```

//复数与double数据相加

```
c3.display( );
```

```
return 0;
```

```
}
```

operator +(c1, 2.5)

相当于:

operator +(c1, Complex (2.5))

对程序的分析:

(1) 如果没有定义转换构造函数, 则此程序编译出错。

(2) 在处理表达式 $c1+2.5$ 时, 编译系统把它解释为:

`operator+(c1, 2.5)`

由于2.5不是Complex类对象, 系统先调用转换构造函数`Complex(2.5)`, 建立一个临时的Complex类对象, 其值为 $(2.5+0i)$ 。上面的函数调用相当于:

`operator+(c1, Complex(2.5))`

将 $c1$ 与 $(2.5+0i)$ 相加, 赋给 $c3$ 。运行结果为: $(5.5+4i)$

(3) 如果把“`c3=c1+2.5;`”改为`c3=2.5+c1;` 程序可以通过编译和正常运行。过程与前相同。

重要结论: 在已定义了相应的转换构造函数情况下, 将运算符“+”函数重载为友元函数, 在进行两个复数相加时, 可以用交换律。

学习目的检测

- 了解运算符重载的概念和意义。
- 掌握运算符重载的限制。
- 掌握流插入和流提取运算符的重载方法。
- 掌握以成员函数的方式重载运算符的方法。
- 掌握以友元函数的方式重载运算符的方法。
- 了解运算符成员函数和友元函数的区别。
- 掌握赋值运算符重载的方法。
- 了解类型转换运算符的重载方法。

Thank you !